Structure-Guided Semi-Supervised Clustering via Evolving Star Network

Zi-Han Dong
Southwest Petroleum University
Chengdu, China
dongzh0830@gmail.com

Wen-Bo Xie*
Southwest Petroleum University
Chengdu, China
wenboxie@swpu.edu.cn

Tao Deng
Southwest Petroleum University
Chengdu, China
dttsytao@gmail.com

Bin Chen
University of Electronic Science and Technology of China
Chengdu, China
202511081726@std.uestc.edu.cn

Tian Zou
Southwest Petroleum University
Chengdu, China
tianz0520@gmail.com

Yan-Li Lee
Xihua University
Chengdu, China
yanlicomplex@gmail.com

Xin Wang
Southwest Petroleum University
Chengdu, China
xinwang@swpu.edu.cn

Jing-hao Liang
Chengdu Technological University
Chengdu, China
2322012104@cdtu.edu.cn

Abstract

Most existing semi-supervised clustering methods rely on dense similarity graphs or global embedding transformations, which incur prohibitive time and space costs and hinder real-world scalability. Moreover, these methods often query user supervision uniformly, leading to redundant and costly annotations. To tackle these challenges, we present EvoStar, a scalable semi-supervised clustering framework guided by a dynamically evolving star network structure. Departing from traditional paradigms, EvoStar introduces a newly designed sparse hierarchical "star" topology that naturally integrates structural cues and supervision. It begins with an unsupervised initialization that organizes data into multi-level star units using nearest-neighbor links and adaptive density peaks. On this foundation, EvoStar actively selects structurally influential nodes for querying and employs a lightweight pruning–grafting mechanism to locally refine the clustering based on user feedback. This design enables implicit constraint propagation through the evolving topology, significantly improving clustering quality with minimal supervision and without relying on external inference models. Extensive experiments on real world datasets demonstrate that EvoStar outperforms baselines under tight constraint budget, while remaining robust to annotation noise. Crucially, EvoStar supports large-scale deployment with $O(n \log n)$ time and $O(n)$ space complexity, making it a compelling choice for interactive clustering. Finally, we provide a detailed theoretical analysis validating the effectiveness of EvoStar under tight constraint budgets. Code: https://github.com/Fly-By-Universe/EvoStar_SIGKDD2026.git

*Wen-Bo Xie is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817628

CCS Concepts

• **Mathematics of computing** → **Cluster analysis**; • **Theory of computation** → *Semi-supervised learning*; *Active learning*.

Keywords

Active clustering, Scalable clustering, Semi-supervised clustering, Pairwise constraints, Knowledge management

ACM Reference Format:
Zi-Han Dong, Wen-Bo Xie, Tao Deng, Bin Chen, Tian Zou, Yan-Li Lee, Xin Wang, and Jing-hao Liang. 2026. Structure-Guided Semi-Supervised Clustering via Evolving Star Network. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817628

1 Introduction

In many large-scale data analysis tasks such as behavioral signal processing [18], visual categorization [17], and user feedback modeling, obtaining full supervision is often prohibitively expensive due to annotation costs or the need for domain expertise. As a practical alternative, constraint-based semi-supervised clustering provides a lightweight way to guide unsupervised clustering using a small number of pairwise constraints, which indicate whether two instances should belong to the same cluster (`must-link`) or different clusters (`cannot-link`). This approach has proven effective in improving clustering quality with minimal human effort.

Despite promising progress [10, 29, 42], current semi-supervised clustering methods still face two fundamental challenges. First, many rely heavily on global propagation mechanisms, e.g., GCPCP [14], FRSMNMF [5], and SPACE [45], which model pairwise constraints through matrix propagation, affinity regularization, or convex optimization. Such designs often incur quadratic to cubic computational complexity (e.g., $O(n^2)$ for affinity learning [27, 38], $O(n^3)$ for spectral decomposition [11, 24]), or even non-polynomial complexity in advanced global optimization frameworks such as SPACE [45], rendering them inefficient or impractical on large

datasets. Second, most methods lack explicit structural guidance for querying: constraints are often collected randomly [43] or based on uncertainty scores alone [15], leading to low efficiency, conflict-prone constraints, and limited scalability. This motivates the development of scalable methods that incorporate structural cues to guide more effective constraint acquisition.

We propose EvoStar, a scalable semi-supervised clustering framework guided by an **Evo**lving **Star** network. EvoStar introduces a dynamic, sparse clustering "skeleton" that evolves via query-guided local updates. It begins by organizing the data into a tailored hierarchical star-shaped structure, where each star unit (a local subgraph) aggregates local neighborhoods and is anchored by an adaptively selected high-density "centers". This structure captures both fine-grained and global structure without relying on domain-specific priors or pretraining. Based on this, a topology-guided querying mechanism prioritizes structurally influential nodes, and user feedback triggers lightweight pruning, grafting and cultivation operations to adaptively refine the evolving star-shaped clustering skeleton. Our key contributions are summarized as follows:

- **Structure-guided clustering skeleton.** We propose a hierarchical star-shaped framework that organizes data into multi-scale local structures centered on adaptive density peaks. This compact representation captures both global and local patterns while supporting fast updates and sparse computation.
- **Topology-aware active refinement.** EvoStar evolves the clustering skeleton through pruning, grafting and cultivation operations guided by user feedback. A novel query strategy prioritizes structurally influential nodes with high deduction potential, leading to efficient supervision.
- **Implicit constraint propagation.** The connectivity of the star units naturally supports logical inference of must-/cannot-links, amplifying the impact of each constraint without relying on external reasoning modules.
- **Scalable and interpretable learning.** EvoStar operates on our sparse graphs with provable $O(n \log n)$ time and $O(n)$ space complexity, making it highly scalable and efficient. Its modular design and transparent update path enable robust performance on large datasets with minimal supervision.
- **Theoretical soundness and empirical validation.** EvoStar offers theoretical guarantees of clustering correctness within finite iterations, and demonstrates state-of-the-art empirical performance in terms of scalability, accuracy, and robustness under minimal supervision across diverse benchmarks.

2 Methodology

This section details the proposed EvoStar framework. We begin by introducing the notations and formal problem definition. Then, we describe the three main components of EvoStar: I. *star network initialization* based on local structure abstraction, II. a *structure-guided query strategy* to select informative nodes for constraint acquisition, and III. a *query-driven structural refinement* process that updates the structure via pruning, grafting and cultivation (i.e., star evolution). The full workflow is illustrated in Figure 1, and the main procedure is summarized in **Algorithm 1**.

Algorithm 1 EvoStar (`Main` Procedure)

Input: Dataset: $X = \{x_i\}_{i=1}^n$; Query budget: $\mathbb{B}$
Output: Star network $\mathcal{S}^\star_{(T)}$ with components $\{S_1, \cdots, S_a\}$

1: $\mathcal{S}^\star_{(0)} \leftarrow$ `StarNInit`(X) ▷ Algorithm 2
2: Initialize priority queue Q based on $\mathcal{S}^\star_{(0)}$ ▷ Eq. (9)
3: $b_+ \leftarrow 0$; $t \leftarrow 0$
4: **while** $b_+ < \mathbb{B}$ **do**
5: $t \leftarrow t + 1$
6: $x^{(t)} \leftarrow$ node with the highest priority from Q
7: $\mathcal{S}^\star_{(t)}, b_+ \leftarrow$ `PGRef`$(\mathcal{S}^\star_{(t-1)}, x^{(t)}, b_+)$ ▷ Algorithm 3
8: **end while**
9: $T \leftarrow t$
10: **return** $\mathcal{S}^\star_{(T)}$

2.1 Problem Setup and Notations

We consider the problem of semi-supervised clustering, where a dataset $X = \{x_i\}_{i=1}^n$ is to be partitioned into an unknown number of clusters. A limited budget of $\mathbb{B}$ pairwise queries is available. Each query asks an oracle whether a pair (x_i, x_j) belongs to the same cluster, returning either a `must-link` or a `cannot-link` constraint. We denote this oracle function as $\tau(x_i, x_j)$.

To model proximity relationships, we define local nearest neighbor operators over arbitrary subsets. For any subset $V \subseteq X$, the nearest neighbor of a data point $x_i \in V$ is defined as:

$$\mathcal{N}_V^{(1)}(x_i) = \arg \min_{x_j \in V \setminus \{x_i\}} \mathtt{d}(x_i, x_j), \tag{1}$$

where $\mathtt{d}(\cdot)$ denotes the pairwise Euclidean distance computed in a normalized feature space. In some iterations, we also use the second-nearest neighbor, defined as:

$$\mathcal{N}_V^{(2)}(x_i) = \arg \min_{x_j \in V \setminus \{x_i, \mathcal{N}_V^{(1)}(x_i)\}} \mathtt{d}(x_i, x_j). \tag{2}$$

At each star evolution iteration $t \in \{1, 2, \dots\}$, EvoStar maintains an intermediate clustering structure as a collection of disjoint star units: $\mathcal{S}^\star_{(t)} = \{S_1, S_2, \dots, S_a\}$, where each S_i is a star-shaped tree centered at node o_i. Collectively, these units form a forest over X. The initial structure $\mathcal{S}^\star_{(0)}$ is constructed in an unsupervised manner, and later constraints may split or refine it. Once the query budget $\mathbb{B}$ is exhausted, EvoStar outputs the final structure $\mathcal{S}^\star_{(T)} (T \le n-1)$, where each star unit S_i corresponds to one final cluster.

2.2 Star Network Initialization

EvoStar begins with an unsupervised initialization that constructs a compact hierarchical star network $\mathcal{S}^\star_{(0)}$, capturing both local coherence and global organization. Unlike prior methods that start from flat clusters or individual points [31, 43], this bottom-up structure serves as a clustering skeleton to be refined via supervision. The procedure is outlined in **Algorithm 2**.

Given a dataset $X = \{x_i\}_{i=1}^n$, we initialize the star network with the node set $\mathcal{V}^\star_{(0)} = X$, edge set $\mathcal{E}^\star_{(0)} = \varnothing$, and representative set $\mathcal{R} \leftarrow \mathcal{V}^\star_{(0)}$. We then iteratively link and compress nodes to build $\mathcal{E}^\star_{(0)}$ until a hierarchical structure with one representative remains.

Local Linking via Nearest Neighbors (lines 8-11). We perform a local linking step to reveal preliminary clusters among the current active nodes. Specifically, we construct a sparse graph $\mathcal{S}^L = (V, E)$ over the current representative set $\mathcal{R}$ by linking each node $x_i \in V$

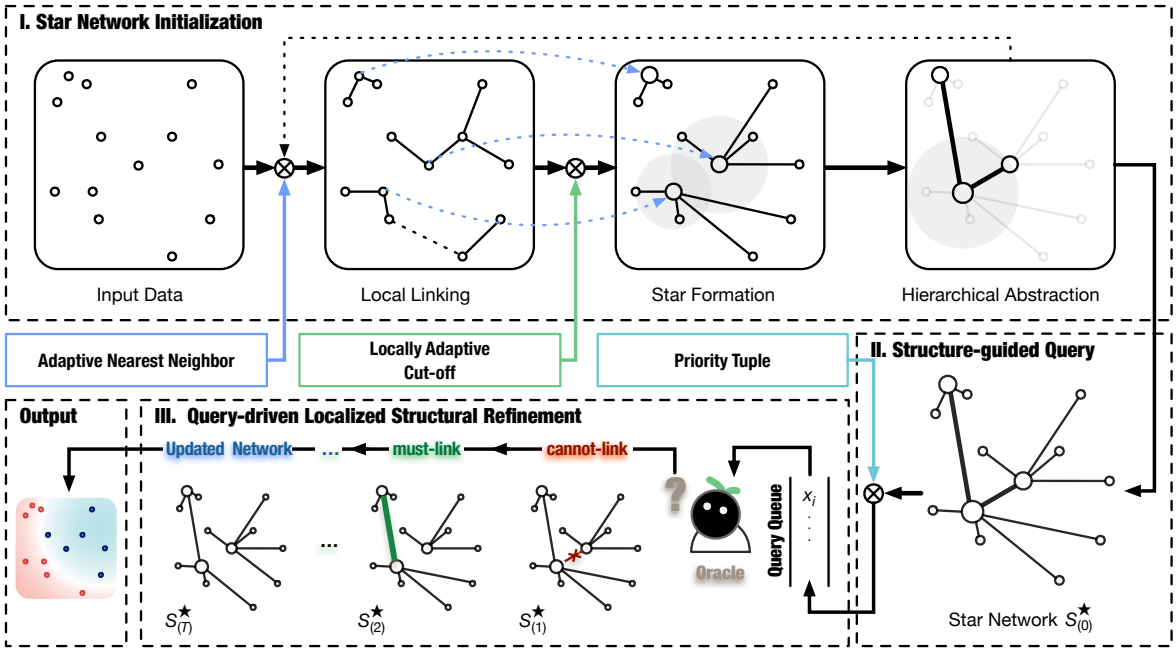


Figure 1: Overall framework of the proposed EvoStar.

to its nearest neighbor:

$$E = \{(x_i, \mathcal{N}_V^{(1)}(x_i)) \mid x_i \in V\}. \tag{3}$$

This step captures spatial coherence within local regions, resulting in multiple connected components that reflect natural groupings.

Occasionally, some nodes form trivial size-2 components when two nodes are each other's nearest neighbors. To avoid such degenerate structures, we apply a lightweight fix: we identify the second-nearest neighbors for both nodes, and connect it to the component with fewer nodes among the two potential targets. This encourages early integration of marginal structures, improving compactness and reducing fragmentation by merging these degenerate size-2 components. As a result, every connected component contains at least three nodes, which is required to make the subsequent representative selection and star refinement well-defined. Lemma 1 states this guarantee.

Lemma 1. *For each component $(V_c, E_c) \subset \mathcal{S}^L$, the lightweight fix ensures that the component satisfies the lower bound $|V_c| \geq 3$.*

Proof. Consider the star network constructed at any level k. Let V_{c1} be a minimal connected component. By the properties of nearest-neighbor graphs, V_{c1} must contain at least one reciprocal pair, implying $|V_{c1}| \geq 2$. In the specific case where $|V_{c1}| = 2$, the algorithm triggers the degeneracy handling mechanism: it selects a node from V_{c1} and establishes a link to its second-nearest neighbor which belongs to a distinct component V_{c2}. Since V_{c2} is also a component within a nearest-neighbor graph, it holds that $|V_{c2}| \geq 2$. Consequently, the cardinality of the newly merged component V_{new} satisfies:

$$|V_{\text{new}}| = |V_{c1}| + |V_{c2}| \geq 2 + 2 = 4.$$

Algorithm 2 `StarNInit`

Input: Dataset: $X = \{x_i\}_{i=1}^n$
Output: Initialized Star Network: $\mathcal{S}_{(0)}^\star$

1: $\mathcal{S}_{(0)}^\star \leftarrow (\mathcal{V}_{(0)}^\star = X, \mathcal{E}_{(0)}^\star = \emptyset)$; $\mathcal{R} \leftarrow \mathcal{V}_{(0)}^\star$
2: **while** $|\mathcal{R}| > 1$ **do**
3: $\quad \mathcal{S}^L, \mathcal{R} \leftarrow$ `LocalEn`$(\mathcal{R})$ ▷ lines 7-19
4: $\quad$ Update $\mathcal{S}_{(0)}^\star$ by $\mathcal{S}_{(0)}^\star \cup \mathcal{S}^L$
5: **end while**
6: **return** $\mathcal{S}_{(0)}^\star$

7: **function** `LocalEn`$(\mathcal{R})$
8: $\quad \mathcal{S}^L \leftarrow (V = \mathcal{R}, E = \{(x_i, \mathcal{N}_V^{(1)}(x_i)) \mid x_i \in V\})$
9: $\quad$ **for each** size-2 component with nodes $\{x_1, x_2\}$ **do**
10: $\quad\quad \left(\left|\text{comp}(\mathcal{N}_V^{(2)}(x_1))\right| < \left|\text{comp}(\mathcal{N}_V^{(2)}(x_2))\right|\right)$? $E.\text{add}(x_1, \mathcal{N}_V^{(2)}(x_1)) : E.\text{add}(x_2, \mathcal{N}_V^{(2)}(x_2))$
11: $\quad$ **end for**
12: $\quad \mathcal{R} \leftarrow \emptyset$
13: $\quad$ **for each** component $(V_c, E_c) \subset \mathcal{S}^L$ **do**
14: $\quad\quad$ Select a representative $x_r \in V_c$ ▷ Eqs. (4-6)
15: $\quad\quad \mathcal{R}.\text{add}(x_r)$
16: $\quad\quad$ Update E by $E \setminus E_c \cup \{(x, x_r) \mid x \in V_c \setminus \{x_r\}\}$
17: $\quad$ **end for**
18: $\quad$ **return** $S^L, \mathcal{R}$
19: **end function**

Consequently, this mechanism eliminates all size-2 components, ensuring that every resulting component satisfies $|V_c| \geq 3$ for the subsequent hierarchical abstraction. □

Star Formation within Components (lines 13-17). To summarize each local region, we select a representative node for each component (V_c, E_c) to form a star centered on it. While centroid- or kernel-based methods [13] are commonly used, they may fail to capture local topology, especially in sparse or manifold data. Inspired by

density peak clustering frameworks [28, 35], we design a structure-guided representative node identification strategy tailored for our evolving star network. Instead of applying a fixed global cutoff as in classical density peak cutoff scheme, we introduce an adaptive, hierarchical version: within each local component, the cutoff distance d_c is set to the α-quantile of all pairwise distances. This locally adaptive thresholding ensures robust and scale-sensitive density estimation, allowing the identification of representative nodes that reflect both local granularity and the global hierarchical topology of the star structure.

Given a connected component (V_c, E_c) and a node $x_i \in V_c$, we define its local density score $\rho(x_i)$ as:

$$\rho(x_i) = \sum_{x_j \in V_c} \kappa\big(\mathrm{d}(x_i, x_j) - d_c\big). \tag{4}$$

Here, $\kappa(\cdot)$ is a indicator function,

$$\kappa(\nu) = \begin{cases} 1, & \nu < 0; \\ 0, & \nu \geq 0. \end{cases} \tag{5}$$

$\mathrm{d}(\cdot)$ is the pairwise distance between nodes; d_c is a dynamic cutoff distance, defined as the $\lceil \alpha\beta \rceil$-th smallest distance among all $\beta = \frac{|V_c|(|V_c|-1)}{2}$ pairs within V_c, with a global threshold $\alpha \in [0, 1]$. The node with the highest $\rho(x_i)$ is selected as the representative x_r,

$$x_r = \arg\max_{x_i \in V_c} \rho(x_i). \tag{6}$$

We then rewire the component into a local star by replacing all internal edges with links pointing to x_r, with x_r as their parent.

This yields a structurally interpretable unit that summarizes the local region with a single central node. Notably, the use of the adaptive cut-off distance in density computation ensures that the selected representative reflects the intrinsic spatial distribution, preserving locality-aware connectivity.

Hierarchical Abstraction (lines 2-5). Representatives from all components form the new active set $\mathcal{R}$, and the process repeats. This recursive abstraction compresses star networks layer by layer until convergence yields the final structure $\mathcal{S}^{\star}_{(0)}$ with a single representative.

To theoretically support the design of our query strategy, we further investigate how the constructed star hierarchy evolves during recursive abstraction. In particular, we aim to understand how representative nodes decay and aggregate mass across layers. Let $\mathcal{R}_k$ denote the representative set at level k.

Lemma 2 (Representative Decay). *Since each representative at level $k+1$ summarizes an expected component of size m from level k, the number of active representatives decreases geometrically across levels:*

$$\mathbb{E}[\,|\mathcal{R}_k|\,] = n \cdot m^{-k}. \tag{7}$$

Proof. The representative set size follows the recurrence $\mathbb{E}[|\mathcal{R}_k|] = \frac{1}{m}\mathbb{E}[|\mathcal{R}_{k-1}|]$. Combining with the initial condition $|\mathcal{R}_0| = n$ yields $\mathbb{E}[|\mathcal{R}_k|] = n/m^k$. □

Lemma 3 (Mass Growth). *Let $\mu(x_r)$ denote the number of descendants represented by a node $x_r \in \mathcal{R}_k$. Then:*

$$\mathbb{E}[\mu(x_r)] = m^k. \tag{8}$$

Proof. The total population n is distributed over the expected representative set size $\mathbb{E}[|\mathcal{R}_k|] = n/m^k$. Therefore, $\mathbb{E}[\mu(x_r)] = n/\mathbb{E}[|\mathcal{R}_k|] = m^k$. □

Consequently, the hierarchy reaches a single representative after at most $k_{\max} = \lceil \log_m n \rceil$ levels in expectation. Since each component contains at least three nodes after the lightweight fix (Lemma 1), the expected component size satisfies $m \geq 3$.

These results characterize a geometric hierarchy: $|\mathcal{R}_k|$ decays while each representative aggregates more descendants as k increases. This provides theoretical justification for our query ranking strategy (Section 2.3), since higher-level representatives influence larger portions of the data.

2.3 Structure-Guided Query Strategy

Guided by the initial hierarchical star structure $\mathcal{S}^{\star}_{(0)}$, EvoStar aims to partition the data into meaningful units by selectively refining the structure. Hence, we propose a structure-aware querying strategy that prioritizes nodes based on their topological role in the current star network. Specifically, each non-center node x_i is assigned a priority tuple:

$$\texttt{priority}(x_i) = \big(\texttt{level}(x_i), \texttt{deg}(x_i), \mathrm{d}(x_i, p(x_i))\big), \tag{9}$$

where $\texttt{level}(x_i)$ is the layer of x_i in the star hierarchy, $\texttt{deg}(x_i)$ is the degree, and $p(x_i)$ is x_i's parent node. The intuition behind each factor is as follows: • **Hierarchy Level:** nodes at higher levels are queried first, as errors near the top propagate to many descendants; • **Connectivity (Degree):** nodes with higher degree are prioritized next, as they are often structural bridges – mistakes here affect multiple regions; • **Distance to Parent:** finally, among otherwise similar nodes, those farther from their parent are considered more likely misattached and thus worth checking.

Refinement Priority. We adopt a lexicographic ordering on these tuples: nodes are first sorted by level (descending), then by degree (descending), and finally by parent distance (descending). At each iteration, the top node x_i is popped and queried against its parent $p(x_i)$. A `must-link` confirms the structure, while a `cannot-link` triggers a localized refinement.

2.4 Localized Structural Refinement

After each query, we update the star network structure to reflect the newly observed constraint. If the oracle returns a `cannot-link` (i.e., x_i and $p(x_i)$ should not belong to the same cluster), we refine the current structure $\mathcal{S}^{\star}_{(t-1)}$ into a new version $\mathcal{S}^{\star}_{(t)}$. To accommodate the new constraint while preserving the hierarchical structure, we perform localized *Pruning* and *Grafting*, optionally followed by *Cultivation* of a new cluster if needed. This localized correction mechanism implicitly enables constraint propagation, as structural consistency naturally spreads through the evolving star network. The full refinement procedure is summarized in **Algorithm 3.**

Pruning (line 4). When processing a `cannot-link` constraint $\tau(x^{(t)}, p(x^{(t)}))$, we treat the current star network $\mathcal{S}^{\star}_{(t-1)}$ by severing the link between node $x^{(t)}$ and its parent $p(x^{(t)})$. If $x^{(t)}$ has children, $x^{(t)}$ and all its descendants are detached to form a new star unit, reflecting the inferred propagation of incompatibility within the unit. This breaks the original star unit, effectively splitting one connected star into two components. Pruning preserves

Algorithm 3 PGRef

Input: Star network: $\mathcal{S}^{\star}_{(t-1)}=(\mathcal{V}^{\star}_{(t-1)}, \mathcal{E}^{\star}_{(t-1)})$; Node: $x^{(t)}$; Current budget: b_+;
Output: Updated star network: $\mathcal{S}^{\star}_{(t)}$, Updated budget: b_+

$\tau(x^{(t)}, p(x^{(t)})) \leftarrow \texttt{Oracle}(x^{(t)}, p(x^{(t)}))$ ▷ Ask Human
$b_+ \leftarrow b_+ + 1$
if $\tau(x^{(t)}, p(x^{(t)}))$ **is** `cannot-link` **then**
  $\mathcal{E}^{\star}_{(t-1)} \leftarrow \mathcal{E}^{\star}_{(t-1)} \setminus \{(x^{(t)}, p(x^{(t)}))\}$
  $\mathbb{O} \leftarrow$ center node of each star unit in $\mathcal{S}^{\star}_{(t-1)}$
  $\mathbb{O} \leftarrow \mathbb{O} \setminus \{$the original star center of $x^{(t)}\}$
  for each o_j **in** $\arg\texttt{sort}_{o_j \in \mathbb{O}} \mathsf{d}(x^{(t)}, o_j)$ **do**
    $\tau(x^{(t)}, o_j) \leftarrow \texttt{Oracle}(x^{(t)}, o_j)$ ▷ Ask Human
    $b_+ \leftarrow b_+ + 1$
    if $\tau(x^{(t)}, o_j)$ **is** `must-link` **then**
      $\mathcal{E}^{\star}_{(t-1)} \leftarrow \mathcal{E}^{\star}_{(t-1)} \cup \{(x^{(t)}, o_j)\}$
      break
    end if
  end for
  if $\forall o_j \in \mathbb{O}, \tau(x^{(t)}, o_j)$ **is** `cannot-link` **then**
    $x^{(t)}$ remains detached as a new center
  end if
end if
$\mathcal{S}^{\star}_{(t)} \leftarrow (\mathcal{V}^{\star}_{(t-1)}, \mathcal{E}^{\star}_{(t-1)})$
return $\mathcal{S}^{\star}_{(t)}, b_+$

the acyclic, star-structured nature of the network and enforces the single-parent invariant (each non-center node has exactly one parent) by removing an invalid parent-child relationship.

Grafting (lines 5-13). After pruning, the detached $x^{(t)}$ and its descendants form an isolated star unit. We reattach this unit to a more suitable location, consistent with the new `cannot-link` constraint. Let $\mathbb{O}$ be the set of star centers in $\mathcal{S}^{\star}_{(t-1)}$, excluding the original star center of $x^{(t)}$. We iterate over candidates $o_j \in \mathbb{O}$ sorted by distance to $x^{(t)}$. For each o_j, a query $\tau(x^{(t)}, o_j)$ determines if $x^{(t)}$ and o_j share the same true cluster. If the oracle returns `must-link`, we attach $x^{(t)}$ to o_j via a new edge $(x^{(t)}, o_j)$, grafting $x^{(t)}$'s sub-star into the unit centered at o_j and merging them. Otherwise, we move to the next candidate.

Cultivation (lines 14-16). If all candidate o_j return a `cannot-link`, $x^{(t)}$ and its descendants are retained as a separate star unit. In this case, $x^{(t)}$ is promoted as the center of a newly formed cluster, and $\mathbb{O}$ is accordingly updated. *Cultivation* process reflects the emergence of a cluster that is structurally incompatible with existing ones.

3 Analysis

Theoretical Analysis and Correctness Guarantees. We analyze EvoStar under a commonly adopted setting where clusters are well-separated and the user oracle provides consistent supervision. That is, the user's clustering intent remains stable throughout the interaction, and the provided constraints are free from internal contradictions. This assumption enables formal reasoning about convergence and query complexity, as in prior theoretical studies of constrained clustering [1, 8, 30, 33, 37, 45].

We define two structured probabilities between nodes x_i and x_j:

- $\mathsf{P}^{\mathsf{H}}_{x_i,x_j} \in [0,1]$: the homogeneity probability, indicating the likelihood that x_i and x_j belong to the same ground-truth cluster;
- $\mathsf{P}^{\mathsf{R}}_{x_i,x_j} = 1 - \mathsf{P}^{\mathsf{H}}_{x_i,x_j}$: the risk probability, indicating the chance of incorrect grouping.

Proposition 1. ***Distance–Probability Relationship.*** *Under the well-separation assumption, the pairwise distance correlates with the risk:*

$$\mathsf{d}(x_i, x_j) \propto \mathsf{P}^{\mathsf{R}}_{x_i,x_j} = 1 - \mathsf{P}^{\mathsf{H}}_{x_i,x_j}. \tag{10}$$

Proposition 1 provides the key bridge between the observed geometry and the latent semantic correctness by associating pairwise distances with the risk probability. Building upon this risk-based formulation, we now analyze how structural uncertainty propagates through EvoStar's hierarchical topology. Specifically, we introduce a representative decay model to quantify the expected semantic gain of querying a level-k representative. This analysis justifies our level-first priority, as the refinement benefit stems from a tradeoff between subtree expansion and path-wise risk.

Representative Decay Model. To provide a theoretical foundation for our query strategy formulated in Eq. (9) and localized correction mechanisms, we introduce the Representative Decay Model. This model offers an interpretation of the hierarchical abstraction process. Specifically, we aim to explain how representative mass growth influences the expected semantic correction gain across recursive abstraction levels, thereby providing a rationale for the design of our sorting mechanism.

We summarize the expected single-step homogeneity at level ℓ by

$$\bar{\pi}_\ell \triangleq \mathbb{E}\Big[\mathsf{P}^{\mathsf{H}}_{x,p(x)} \mid x \in \mathcal{R}_{\ell-1}\Big], \tag{11}$$

where the expectation is taken over all nodes at level $\ell-1$. Intuitively, $\bar{\pi}_\ell$ measures the average reliability of semantic propagation when moving from level $\ell-1$ to level ℓ.

The path-wise homogeneity $\mathsf{P}^{\mathsf{H}}_{x,x_r}$ depends on the cumulative reliability along the ancestor chain. Since the exact dependence structure across different levels is generally intractable and the number of ground-truth clusters K is unknown, we adopt a standard mean-field approximation that treats the single-step reliabilities along the path as weakly dependent in expectation. Consequently, the expected path-wise homogeneity is approximated by the product of level-wise expectations:

$$\mathbb{E}\big[\mathsf{P}^{\mathsf{H}}_{x,x_r}\big] \approx \prod_{\ell=1}^{k} \bar{\pi}_\ell. \tag{12}$$

Then, by $\mathsf{P}^{\mathsf{R}}_{x,x_r} = 1 - \mathsf{P}^{\mathsf{H}}_{x,x_r}$ and linearity of expectation,

$$\mathbb{E}\big[\mathsf{P}^{\mathsf{R}}_{x,x_r}\big] = 1 - \mathbb{E}\big[\mathsf{P}^{\mathsf{H}}_{x,x_r}\big] \approx 1 - \prod_{\ell=1}^{k} \bar{\pi}_\ell. \tag{13}$$

We determine the distribution of descendant nodes within the subtree of a level-k representative. Due to the recursive abstraction process, only a fraction $1/m$ of nodes at level j propagate to the subsequent level $j+1$. Consequently, the expected number of nodes that remain at level j is given by the difference in population sizes between consecutive levels:

$$\mathbb{E}[|\mathcal{R}_j|] - \mathbb{E}[|\mathcal{R}_{j+1}|] = \frac{n}{m^j} - \frac{n}{m^{j+1}} = \frac{n}{m^j}\left(1 - \frac{1}{m}\right). \tag{14}$$

At level k, there are $\mathbb{E}[|\mathcal{R}_k|] = n/m^k$ active representatives. Assuming a uniform distribution of data points, the expected number of level-j descendants associated with a representative $x_r \in \mathcal{R}_k$ is:

$$\mathbb{E}[\mu_j(x_r)] = \frac{\mathbb{E}[|\mathcal{R}_j|] - \mathbb{E}[|\mathcal{R}_{j+1}|]}{\mathbb{E}[|\mathcal{R}_k|]} = m^{k-j}\left(1 - \frac{1}{m}\right). \tag{15}$$

Next, we quantify the correction probability. Based on the mean-field abstraction model, the semantic validity of a link between a node at level j and its representative at level k depends on the cumulative reliability of the path $j \to j+1 \to \cdots \to k$. Let $\bar{\pi}_\ell$ denote the expected single-step homogeneity at level ℓ. The homogeneity is the product of step-wise probabilities:

$$\mathbb{E}[\mathrm{P}^{\mathrm{H}}_{j\to k}] \approx \prod_{\ell=j+1}^{k} \bar{\pi}_\ell. \tag{16}$$

Correspondingly, the expected risk probability that the current structure entails errors for a node at level j is:

$$\mathbb{E}[\mathrm{P}^{\mathrm{R}}_{j\to k}] = 1 - \mathbb{E}[\mathrm{P}^{\mathrm{H}}_{j\to k}] \approx 1 - \prod_{\ell=j+1}^{k} \bar{\pi}_\ell. \tag{17}$$

The total Expected Semantic Gain depends on the net difference between correcting wrong nodes and disturbing correct nodes. We define the exact risk mass and homogeneity mass by summing over all levels $j < k$:

$$\begin{aligned} \mathcal{M}_{\text{safe}}(k) &= \sum_{j=0}^{k-1} \mathbb{E}[\mu_j(x_r)] \cdot \mathbb{E}[\mathrm{P}^{\mathrm{H}}_{j\to k}], \\ \mathcal{M}_{\text{risk}}(k) &= \sum_{j=0}^{k-1} \mathbb{E}[\mu_j(x_r)] \cdot \mathbb{E}[\mathrm{P}^{\mathrm{R}}_{j\to k}]. \end{aligned} \tag{18}$$

Substituting the mass and probability terms yields the defined components leading to the net objective $\mathcal{G}(k) = \mathcal{M}_{\text{safe}}(k) - \mathcal{M}_{\text{risk}}(k)$:

$$\begin{aligned} \mathcal{G}(k) &= \sum_{j=0}^{k-1} m^{k-j}\left(1-\frac{1}{m}\right)\left[\left(\prod_{\ell=j+1}^{k}\bar{\pi}_\ell\right) - \left(1 - \prod_{\ell=j+1}^{k}\bar{\pi}_\ell\right)\right] \\ &= \sum_{j=0}^{k-1} m^{k-j}\left(1-\frac{1}{m}\right)\left(2\prod_{\ell=j+1}^{k}\bar{\pi}_\ell - 1\right). \end{aligned} \tag{19}$$

For $m \geq 3$, the summation is dominated by terms with the largest path lengths (i.e., small j, particularly $j = 0$). This characterizes the asymptotic growth behavior. The dominant contribution arises from the deepest hierarchy level, leading to:

$$\mathcal{G}(k) = \Theta\left(m^k\left(2\prod_{\ell=1}^{k}\bar{\pi}_\ell - 1\right)\right). \tag{20}$$

This confirms that the net gain function is driven by the competition between the geometrically growing component mass and the decaying structural correctness. Guided by this Representative Decay Model, querying representative nodes at higher levels yields exponentially larger potential semantic gains. This theoretical insight formally justifies placing the hierarchy level at the forefront of our query prioritization. Furthermore, to differentiate active representatives within the same level, we introduce a degree-based heuristic, prioritizing nodes with a larger number of immediate descendants to maximize the impact of local rectifications.

Finally, Proposition 1 establishes that a larger spatial distance between a node and its parent inherently carries a higher risk probability. Consequently, node pairs exhibiting larger parent-child distances are prioritized for refinement queries. Conversely, when a subtree is detached via the pruning operation, the homogeneity probability dictates that it should be grafted onto the nearest alternative star center. Combining these theoretical properties and spatial heuristics, we formulate the holistic triplet sorting strategy in Eq. (9), which organically guides the sequential execution of the pruning, grafting, and cultivation operations.

Convergence and Query Budget Upper Bound. For a dataset with n items and K ground-truth clusters, EvoStar establishes a maximum query bound of $nK - (K^2+K)/2$ for fully accurate clustering. During the network initialization phase (Section 2.2), establishing the latent clusters requires identifying K root nodes from the $n-1$ non-root candidates. For each newly discovered root, the *Cultivation* operation involves querying all previously identified roots, accumulating $\sum_{i=0}^{K-1} i = \frac{K(K-1)}{2}$ queries. For the remaining $n-K$ nodes, identifying their cluster assignment requires at most K queries each. Once the cluster assignment of every node is determined, no further structural correction is required, indicating that the active clustering process has converged. Combining two phases, the total pairwise query budget is strictly bounded by:

$$\frac{K(K-1)}{2} + (n-K)K = nK - \frac{K^2+K}{2}.$$

Time complexity. We now analyze the time complexity of EvoStar by evaluating its three major components.

- **`StarNInit`:** EvoStar begins by linking each node to its nearest neighbor, producing a sparse graph with $O(n \log n)$ time via efficient neighbor search. The resulting sparse graph naturally decomposes into multiple small connected components. Each component is then locally refactored into a star by selecting a center based on density peak detection. Crucially, density is computed only with respect to the local neighborhood of the previous-round star center, avoiding global distance computations. If there are c components of size m on average, this step takes $O(m^2 c) = O(n)$ time per round. After star restructuring, all newly selected centers serve as representative nodes for the next round of merging. A fresh nearest neighbor graph is constructed among these centers, triggering the next compression step. In the worst-known case [36], each merged component contains only three nodes. This leads to a geometric series of nearest neighbor linking costs: $n \log n + \frac{n}{3}\log\frac{n}{3} + \cdots = O(n \log n)$.
- **`Priority-based Query`:** Based on Eq.(9), each non-center node is assigned a structural priority tuple, and the entire set is sorted accordingly. This leads to a time complexity of $O(n \log n)$.
- **`PGRef`:** In each iteration, every non-center node may undergo a prune operation once and attempt at most $K-1$ grafting trials to reconnect to other centers, where K is the number of clusters. Each trial involves querying the oracle. With n nodes and at most $n-1$ iterations, the total cost is bounded by $O(Kn)$.

Based on the above analysis, the overall time complexity of EvoStar is $O(n \log n) + O(n \log n) + O(Kn) = O(n \log n)$.

Space Complexity. At all stages of EvoStar, only local structures such as nearest neighbors, component assignments, and sparse star connections are stored. No global adjacency matrix or constraint table is maintained. Therefore, the space usage grows linearly with the number of data points, yielding an overall complexity of $O(n)$.

4 Experiment

4.1 Settings

Datasets. We evaluate EvoStar on diverse benchmarks, covering classic challenges and real-world scenarios. This includes seven UCI datasets [25] of varying scales and domains (e.g., sensor readings, acoustic signals), as well as five large-scale datasets widely used in interactive AI applications: EEG (149,80 brainwave recordings

Table 1: Description of datasets

Dataset	# Samples	# Attr.	# Classes	Dataset	# Samples	# Attr.	# Classes
tae	151	5	3	sonar	208	60	2
ionosphere	351	34	2	musk	476	166	2
karhunen	2,000	64	10	zernike	2,000	47	10
segment	2,310	19	7	EEG	14,980	14	2
MNIST	70,000	784	10	ALOI	108,000	128	1,000
CoverType	581,012	54	7	HAR	2,259,597	6	7

for mental state prediction) [3], MNIST (70,000 handwritten digits for visual recognition systems) [21], ALOI (108,000 3D object images across 1,000 classes for interactive recognition) [9], CoverType (581,012 forestry records for environmental modeling) [19], and HAR (2.26 million time-series samples for activity tracking in mobile applications) [32]. These datasets span domains such as visual recognition, time-series modeling, and human-centered feedback, providing a comprehensive testbed for evaluating constraint-based clustering in semi-supervised settings. Dataset statistics are summarized in Table 1.

Baselines. We compare EvoStar with eight semi-supervised clustering methods, covering classical heuristics and recent scalable frameworks. **FFQS** [2] adopts farthest-point querying to explore new regions. **COBRA** [34] and **COBRAS**[33], merge or split super-instances to refine cluster boundaries. **ADPE** [30] builds a global density-peak tree to guide structural queries. **SPACE** [45] leverages ensemble-guided pair selection, while**A3S** [6] performs query-driven refinement based on cluster purity. **SSSE** [43] achieves a $O(n \log n)$ time complexity via sampling but requires $O(n^2)$ memory. We also include **ACMC** [37], a SOTA method that builds a sparse diffusion tree to guide structure-aware label propagation, achieving scalable clustering with $O(n^2/\beta)$ time and $O(n)$ space. Crucially, while ACMC and ADPE rely on iterative label propagation, EvoStar fundamentally departs from this paradigm. By replacing explicit label assignment with dynamic graph connectivity maintained through strictly $O(1)$ local topological updates—namely pruning, grafting, and cultivation—EvoStar effectively avoids the inherent overhead of iterative propagation, leading to a substantial enhancement in computational efficiency.

Implementations. All experiments are conducted on a standard PC with a 4.2GHz CPU and 32.0 GB RAM. We evaluate all methods under consistent constraint budgets using Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI), widely adopted in semi-supervised clustering [6, 15, 43, 44]. For each baseline, we use public source code and follow the default parameter settings recommended in original papers, empirically validated in prior studies. EvoStar also uses fixed parameters across all datasets without dataset-specific tuning, as discussed in the parameter sensitivity analysis. All results are averaged over fifty independent runs.

4.2 Results

Exp. 1: Parameter Analysis of α. EvoStar involves only a single hyperparameter α, which controls the percentage threshold for adaptive distance truncation during star network initialization. To evaluate its impact, we design a normalized bias metric. For each dataset i, we record the number of constraints required to reach ARI thresholds of 0.6, 0.7, and 0.8, and take the average as $\theta^i(\alpha)$.

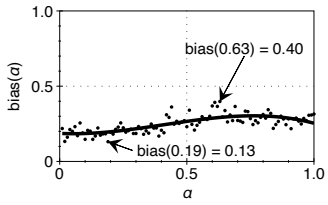


Figure 2: Impact of parameter α on constraint efficiency.

The scaled bias is then defined as,

$$\text{bias}^i(\alpha) = \frac{\theta^i(\alpha) - \theta^i_{\min}}{\theta^i_{\min}}, \tag{21}$$

where $\theta^i_{\min}$ is the minimum number of constraints over all tested values. Figure 2 reports the average bias across datasets for various α values. The results indicate limited sensitivity to α. We recommend α=0.19 as it yields the most stable performance across different targets and datasets.

Exp. 2: Clustering quality under limited budget. Table 2 presents the clustering performance of all methods under varying constraint budgets, measured by Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI). EvoStar delivers superior or highly competitive performance by effectively utilizing limited queries via structure-aware prioritization and refinement. Conversely, baselines like SSSE often saturate early due to rigid mechanisms (e.g., query-driven split/merge) that hinder adaptability. While COBRA's super-instance mechanism excels in extremely low-budget settings, its fixed structure limits performance gains from later queries. Our EvoStar exhibits steady improvements with additional queries. Its progressive tree evolution enables dynamic reorganization of clusters based on topological roles, enabling finer-grained corrections. ACMC leverages sparse diffusion trees, also performs competitively across many datasets. However, its performance trails EvoStar under low-budget regimes, as EvoStar's priority-driven correction mechanism helps maximize the utility of each constraint. Additionally, to intuitively demonstrate the performance gap, we visualize the evolution of ARI and NMI scores under varying constraint budgets in Figure 5 of the Appendix.

Exp.3: Constraint Robustness. We evaluate the robustness of EvoStar under supervision noise on two representative datasets, namely sonar (small-scale) and karhunen (medium-scale), by randomly flipping 2% of oracle responses to simulate real-world annotation errors. This follows previous studies [4, 20], which report typical human error rates between 1% and 2%. We compare EvoStar against ACMC [37]. As shown in Figure 3, each ARI curve represents the average of 100 runs under varying constraint budgets. EvoStar achieves higher ARI scores than ACMC under noisy conditions. We also visualize the performance gap between noise-free and noisy settings as shaded regions, which remain narrow across all budgets. This suggests that EvoStar's evolving structure can absorb early-stage errors and recover effectively over time. In contrast, ACMC exhibits a larger gap and flatter improvement, indicating higher sensitivity to supervision noise. These results confirm EvoStar's stability in realistic active clustering scenarios with annotation errors. To further assess robustness, we evaluate performance under 1%, 5%, and 10% error rates in Figure 6 of the Appendix to simulate varying annotation qualities.

Table 2: Clustering quality on small-sized datasets under different query budgets. The left subtable reports Adjusted Rand Index (ARI), and the right subtable reports Normalized Mutual Information (NMI). Highlighted values represent the highest score per budget level.

Dataset	#Queries	Adjusted Rand Index (ARI)								
		EvoStar	ACMC	FFQS	COBRA	COBRAS	ADPE	A3S	SSSE	SPACE
tae	100	**0.338**	0.145	0.209	0.329	0.227	0.153	0.050	0.059	0.014
	150	**0.650**	0.483	0.557	0.329	0.267	0.317	0.050	0.075	0.148
	200	**1.000**	0.700	0.565	0.329	0.435	0.624	0.050	0.056	0.133
sonar	100	**0.674**	0.398	0.237	0.605	0.450	0.159	0.040	0.016	0.007
	150	**0.842**	0.611	0.364	0.605	0.651	0.504	0.040	0.017	0.007
	200	**0.981**	0.799	0.715	0.605	0.731	0.715	0.040	0.026	0.007
ionosphere	100	0.615	0.640	0.359	**0.712**	0.645	0.615	0.115	0.038	0.062
	150	**0.751**	0.681	0.339	0.712	0.664	0.691	0.166	0.038	0.062
	200	**0.899**	0.751	0.542	0.712	0.732	0.711	0.166	0.038	0.205
musk	100	**0.385**	0.021	0.035	0.219	0.252	0.009	0.009	0.007	0.076
	200	**0.509**	0.254	0.047	0.219	0.360	0.002	0.044	0.007	<0.001
	400	**0.823**	0.778	0.491	0.219	0.603	0.552	0.554	0.008	0.038
karhunen	200	**0.825**	0.584	0.577	0.757	0.681	0.160	0.052	0.752	0.605
	400	**0.898**	0.627	0.640	0.757	0.778	0.280	0.059	0.752	0.605
	600	**0.926**	0.794	0.626	0.757	0.840	0.573	0.138	0.750	0.605
zernike	200	**0.660**	0.400	0.450	0.616	0.494	0.005	0.178	0.496	0.364
	400	**0.710**	0.431	0.475	0.616	0.578	0.022	0.230	0.525	0.367
	600	**0.757**	0.460	0.419	0.616	0.615	0.070	0.279	0.512	0.366
segment	200	**0.798**	0.618	0.635	0.758	0.719	0.193	0.555	0.580	0.503
	400	**0.870**	0.743	0.662	0.758	0.784	0.253	0.574	0.605	0.503
	600	**0.903**	0.766	0.536	0.758	0.810	0.395	0.574	0.507	0.503

Dataset	#Queries	Normalized Mutual Information (NMI)								
		EvoStar	ACMC	FFQS	COBRA	COBRAS	ADPE	A3S	SSSE	SPACE
tae	100	0.330	0.345	0.213	0.290	0.221	0.230	**0.382**	0.105	0.050
	150	**0.591**	0.534	0.548	0.290	0.244	0.370	0.382	0.127	0.202
	200	**1.000**	0.674	0.535	0.290	0.381	0.664	0.382	0.126	0.172
sonar	100	**0.566**	0.434	0.181	0.502	0.365	0.261	0.272	0.033	0.068
	150	**0.759**	0.595	0.283	0.502	0.543	0.408	0.272	0.032	0.068
	200	**0.961**	0.750	0.610	0.502	0.637	0.630	0.272	0.048	0.068
ionosphere	100	0.523	0.552	0.278	**0.597**	0.537	0.554	0.271	0.050	0.083
	150	**0.648**	0.629	0.261	0.597	0.555	0.637	0.296	0.051	0.083
	200	**0.818**	0.691	0.443	0.597	0.620	0.655	0.296	0.051	0.220
musk	100	**0.341**	0.174	0.041	0.230	0.249	0.067	0.209	0.008	0.149
	200	**0.445**	0.293	0.031	0.230	0.311	0.065	0.234	0.008	0.094
	400	**0.753**	0.732	0.387	0.230	0.524	0.559	0.441	0.010	0.158
karhunen	200	**0.845**	0.760	0.678	0.804	0.743	0.325	0.523	0.820	0.645
	400	**0.898**	0.805	0.718	0.804	0.798	0.469	0.532	0.815	0.645
	600	**0.926**	0.904	0.715	0.804	0.845	0.700	0.552	0.819	0.645
zernike	200	**0.715**	0.561	0.557	0.682	0.593	0.109	0.529	0.630	0.479
	400	**0.756**	0.606	0.571	0.682	0.640	0.198	0.560	0.647	0.484
	600	**0.800**	0.640	0.543	0.682	0.672	0.291	0.585	0.649	0.484
segment	200	**0.821**	0.721	0.700	0.800	0.760	0.317	0.647	0.685	0.596
	400	**0.873**	0.830	0.719	0.800	0.805	0.365	0.663	0.693	0.597
	600	**0.905**	0.838	0.638	0.800	0.829	0.558	0.663	0.647	0.597

Table 3: Clustering performance (ARI and NMI) vs. query budget on large-scale datasets. 'n/a' and 'oom' indicate the baseline failed to return results within 48h or exceeded memory limits.

Dataset	#Queries	Adjusted Rand Index (ARI)				Normalized Mutual Information (NMI)			
		EvoStar	ACMC	COBRA	COBRAS	EvoStar	ACMC	COBRA	COBRAS
EEG	1,000	**0.199**	<0.001	0.017	0.084	**0.147**	0.018	0.011	0.056
	5,000	**0.463**	0.181	0.017	0.306	**0.364**	0.154	0.011	0.230
	10,000	**0.655**	0.545	0.017	0.447	**0.544**	0.440	0.011	0.349
MNIST	1,000	**0.862**	0.430	0.746	n/a	**0.855**	0.623	0.762	n/a
	5,000	**0.903**	0.478	0.746	n/a	**0.891**	0.680	0.762	n/a
	10,000	**0.927**	0.578	0.746	n/a	**0.914**	0.775	0.762	n/a
ALOI	10,000	**0.088**	0.006	0.071	oom	**0.674**	0.360	0.611	oom
	20,000	**0.110**	0.006	0.071	oom	**0.702**	0.303	0.611	oom
	50,000	**0.171**	0.006	0.071	oom	**0.747**	0.370	0.611	oom
CoverType	10,000	**0.505**	0.119	0.124	oom	**0.479**	0.206	0.179	oom
	20,000	**0.589**	0.152	0.124	oom	**0.550**	0.231	0.179	oom
	50,000	**0.707**	0.196	0.124	oom	**0.657**	0.258	0.179	oom
HAR	2,000	**0.610**	0.483	0.498	oom	**0.692**	0.653	0.581	oom
	10,000	**0.685**	0.475	0.498	oom	**0.724**	0.652	0.581	oom
	50,000	**0.737**	0.459	0.498	oom	**0.746**	0.646	0.581	oom

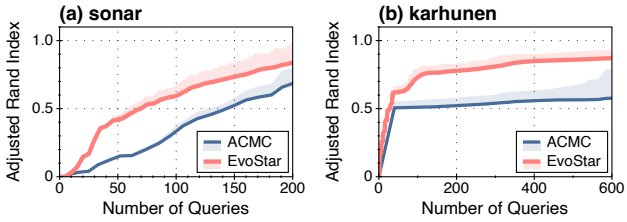
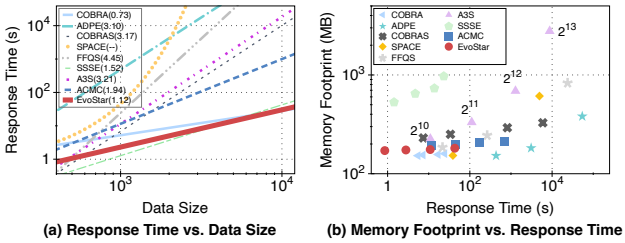


Figure 3: Robustness to noisy constraints on sonar and karhunen. Shaded regions show ARI differences between noise-free and noisy settings.

Exp. 4: Scalability. Table 3 evaluates performance on large-scale datasets up to 2.2 million instances. EvoStar consistently achieves the highest ARI/NMI across all budgets, significantly outperforming scalable baselines. Notably, on CoverType, EvoStar reaches 0.707 ARI with 50k constraints, surpassing ACMC and COBRA by over 0.5.



(a) Response Time vs. Data Size (b) Memory Footprint vs. Response Time

Figure 4: Scalability evaluation on synthetic datasets of increasing sizes.

Similar gains are observed on the HAR dataset (0.737 ARI). While most baselines fail to execute within time (n/a) or memory (oom) limits, EvoStar maintains high clustering quality and efficiency under limited supervision, proving its robustness for massive, complex data.

We further validate scalability using synthetic datasets of increasing sizes, ranging from 2^{10} to 2^{13}. As shown in Figure 4, EvoStar consistently demonstrates fast response times and low memory footprint across all data scales. In subplot (a), EvoStar maintains a near-linear growth trend in response time as data size increases, with a fitted power exponent of 1.12. In subplot (b), both EvoStar and ACMC occupy the lower-left region of the memory–time plane, indicating superior overall efficiency. While COBRA exhibits low memory footprint and response time due to its super-instance compression, this mechanism causes early performance saturation by restricting cluster refinement as constraints increase. In contrast, EvoStar maintains high clustering quality with low computational costs. Although SSSE offers competitive response times, its significantly higher memory usage limits applicability in constrained environments. Overall, EvoStar achieves the superior trade-off among runtime, memory efficiency, and quality, providing a practical, scalable solution for large-scale semi-supervised clustering.

Table 4: AUIC@n scores of EvoStar and its ablated variants across small- and medium-sized datasets.

Datasets	EvoStar	w/o-Graft	w/o-Peak	w/o-Sort
tae	**0.28**	0.18	0.22	0.13
sonar	**0.64**	0.14	0.52	0.12
ionosphere	**0.78**	0.70	0.64	0.33
musk	**0.55**	0.19	0.50	0.12
segment	**0.93**	0.67	0.89	0.36
karhunen	**0.94**	0.74	0.81	0.29
zernike	**0.78**	0.56	0.63	0.16
MEAN	**0.70**	0.45	0.60	0.22

Exp. 5: Ablation study. To assess the contribution of each key strategy in EvoStar, we conduct a comprehensive ablation study on all small- and medium-sized datasets. We consider three simplified variants of EvoStar, each disabling a core strategy: • **w/o-Graft**: disables the grafting mechanism, preventing the reattachment of pruned substructures into the evolving clustering tree. • **w/o-Peak**: replaces the local density peak-based center selection during initialization with random selection, removing structure-aware compression. • **w/o-Sort**: removes the priority-based query selection strategy, selecting constraint pairs randomly regardless of topological or uncertainty-based cues.

We evaluate EvoStar and its variants using the AUIC@n (Area Under the Interaction Curve), which quantifies clustering performance under a fixed query budget n. Analogous to AUC in ROC analysis, AUIC@n measures the cumulative accuracy relative to the number of provided constraints. Let a_i denote the clustering accuracy (e.g., ARI) after the i-th query; the score is defined via trapezoidal integration:

$$\text{AUIC@}n = \frac{1}{n}\sum_{i=0}^{n-1}\frac{a_i + a_{i+1}}{2}. \tag{22}$$

A higher AUIC@n signifies superior interaction efficiency, indicating that the method achieves high accuracy both rapidly in early stages and consistently throughout the query process.

Table 4 reports the AUIC@n scores. Across all datasets, EvoStar achieves the highest scores, indicating the importance of all core strategies. • w/o-Graft results in moderate performance degradation, as the inability to reintegrate previously pruned substructures limits EvoStar's flexibility to refine past decisions based on new constraints. • w/o-Peak triggers sharp drops, especially on complex datasets (e.g., sonar, musk), highlighting density peaks' role in establishing hierarchical scaffolds. • w/o-Sort performs the worst, with the lowest average AUIC score (0.22). Without prioritization, query usage becomes inefficient as the algorithm fails to focus on uncertain or impactful regions.

These results demonstrate that EvoStar's performance gains stem from the synergy of its three strategies: principled initialization, targeted querying, and adaptive structure evolution. Disabling any one of them weakens this synergy and reduces the framework's ability to effectively leverage limited supervision.

5 Related Work

Semi-supervised clustering incorporates limited supervision, typically in the form of pairwise constraints, to guide data partitioning. Early semi-supervised clustering methods, such as COPK-Means [39] and SCHAIN [23], directly embed must-link and cannot-link constraints into the clustering objective. However, these methods often propagate supervision only locally, limiting their global influence on structure discovery. Metric learning approaches enhance constraint propagation by adapting the similarity space, either through Mahalanobis distance learning or low-rank approximations [16, 40]. While effective in reshaping the data geometry, such methods typically involve high computational overhead and face difficulty scaling to non-linear or large-scale manifolds. To better capture local structure, graph-based models have been widely adopted, including spectral clustering [22, 41], label propagation [31], and anchor-based approximations [12]. Yet, their reliance on a fixed, pre-constructed similarity graph introduces fragility (initial graph errors often propagate uncontrollably), undermining supervision effectiveness.

Active clustering aims to mitigate this by selectively querying informative constraints[7, 30, 44], typically based on uncertainty or diversity heuristics. Some recent efforts leverage graph diffusion [7] or random walk priors to score candidate queries, such as ACMC [37], which constructs a diffusion-based clustering tree to propagate constraints globally via topological distances. This structure-aware mechanism improves early-stage accuracy and reduces redundant queries. However, like most existing methods, its static initialization limits adaptability when facing evolving supervision. In contrast, EvoStar dynamically evolves its clustering structure in response to new constraints, enabling progressive refinement and sustained performance improvements throughout the interaction process.

Density-based clustering, exemplified by Density Peak Clustering (DPC) [28], excels at identifying arbitrarily shaped clusters via global density peaks. Semi-supervised extensions, such as SS-DBSCAN [26] and ADPE [30], typically apply constraints after unsupervised structure discovery, either by reassigning points or adjusting boundary labels. However, these approaches operate on fixed cluster structures and lack fine-grained structural adaptability.

6 Conclusion

The proposed EvoStar introduces a novel structure-guided semi-supervised clustering framework that embeds an implicit constraint propagation mechanism within a multi-level star network. Each local star serves as a structural unit where must-link constraints can propagate transitively among member nodes, and their interactions via pruning and grafting refine the global structure. This design enables the co-evolution of structure and supervision, allowing human feedback to dynamically reshape the clustering process. As a result, EvoStar amplifies the effect of sparse supervision, confirming or rejecting one link often clarifies entire regions of the structure. Looking ahead, we will extend EvoStar to dynamic settings with continual feedback, where user criteria may drift over time. We also plan to incorporate lightweight learnable representations to better adapt the evolving structure to heterogeneous data.

Acknowledgments

This work is supported by the Sichuan Science and Technology Program [No. 2024NSFSC1464] and the Scientific Research Fund of Key Lab of Internet Natural Language Processing of Sichuan Provincial Education Department [No. INLP202601] and the Natural Science Starting Project of SWPU [No. 2023QHZ010].

References

[1] Liang Bai, Jiye Liang, and Yunxiao Zhao. 2023. Self-Constrained Spectral Clustering. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 4 (2023), 5126–5138.

[2] Sugato Basu, Arindam Banerjee, and Raymond J Mooney. 2004. Active semi-supervision for pairwise constrained clustering. In *Proceedings of the 2004 SIAM International conference on Data Mining*. 333–344.

[3] Henri Begleiter. 1995. EEG Database. https://doi.org/10.24432/C5TS3D

[4] Arijit Biswas and David Jacobs. 2014. Active image clustering with pairwise constraints from humans. *International Journal of Computer Vision* 108, 1 (2014), 133–147.

[5] Guosheng Cui, Ruxin Wang, Dan Wu, and Ye Li. 2024. Semi-supervised Multi-view Clustering based on NMF with Fusion Regularization. *ACM Transactions on Knowledge Discovery from Data* 18, 6 (2024), 1–26.

[6] Xun Deng, Junlong Liu, Han Zhong, Fuli Feng, Chen Shen, Xiangnan He, Jieping Ye, and Zheng Wang. 2024. A3S: A General Active Clustering Method with Pairwise Constraints. In *International Conference on Machine Learning*. 10488–10505.

[7] Xun Fu, Wen-Bo Xie, Bin Chen, Tao Deng, Tian Zou, and Xin Wang. 2024. ACDM: An Effective and Scalable Active Clustering with Pairwise Constraint. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 643–652.

[8] Yifan Fu, Bin Li, Xingquan Zhu, and Chengqi Zhang. 2011. Do they belong to the same class: active learning by querying pairwise label homogeneity. In *Proceedings of the 20th ACM International Conference on Information and Knowledge Management*. 2161–2164.

[9] Jan-Mark Geusebroek, Gertjan J Burghouts, and Arnold WM Smeulders. 2005. The Amsterdam library of object images. *International Journal of Computer Vision* 61, 1 (2005), 103–112.

[10] Germán González-Almagro, Daniel Peralta, Eli De Poorter, José-Ramón Cano, and Salvador García. 2025. Semi-supervised constrained clustering: an in-depth overview, ranked taxonomy and future research directions. *Artificial Intelligence Review* 58, 5 (2025), 157.

[11] Germán González-Almagro, Pablo Sánchez-Bermejo, Juan Luis Suarez, José-Ramón Cano, and Salvador García. 2024. Semi-supervised clustering with two types of background knowledge: Fusing pairwise constraints and monotonicity constraints. *Information Fusion* 102 (2024), 102064.

[12] Wei Guo, Zhe Wang, and Wei Shao. 2026. Structure anchor graph learning for multi-view clustering. *Pattern Recognition* 170 (2026), 111880.

[13] Avisek Gupta and Swagatam Das. 2022. Transfer Clustering Using a Multiple Kernel Metric Learned Under Multi-Instance Weak Supervision. *IEEE Transactions on Emerging Topics in Computational Intelligence* 6, 4 (2022), 828–838.

[14] Guoliang He, Dawei Jin, Wenjun Jiang, Zongkun Zhao, Lifang Dai, Zhiwen Yu, and C.L. Philip Chen. 2024. Efficient semi-supervised clustering with pairwise constraint propagation for multivariate time series. *Information Sciences* 681 (2024), 121233.

[15] Shizhe Hu, Binyan Tian, Weibo Liu, and Yangdong Ye. 2025. Self-supervised Trusted Contrastive Multi-view Clustering with Uncertainty Refined. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 16 (2025), 17305–17313.

[16] Yuheng Jia, Hui Liu, Junhui Hou, and Sam Kwong. 2020. Pairwise Constraint Propagation With Dual Adversarial Manifold Regularization. *IEEE Transactions on Neural Networks and Learning Systems* 31, 12 (2020), 5575–5587.

[17] Lianghai Jin and Liyuan Guo. 2025. Facial Expression Recognition Based on Feature Representation Learning and Clustering-Based Attention Mechanism. *IEEE Transactions on Biometrics, Behavior, and Identity Science* 7, 2 (2025), 182–194.

[18] Yosuke Kashiwagi, Hayato Futami, Emiru Tsunoo, Siddhant Arora, and Shinji Watanabe. 2025. Hypothesis Clustering and Merging: Novel MultiTalker Speech Recognition with Speaker Tokens. In *2025 IEEE International Conference on Acoustics, Speech and Signal Processing*. 1–5.

[19] Ari Kobren, Nicholas Monath, Akshay Krishnamurthy, and Andrew McCallum. 2017. A Hierarchical Algorithm for Extreme Clustering. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 255–264.

[20] Neeraj Kumar, Alexander C Berg, Peter N Belhumeur, and Shree K Nayar. 2009. Attribute and simile classifiers for face verification. In *2009 IEEE 12th International Conference on Computer Vision*. 365–372.

[21] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86, 11 (1998), 2278–2324.

[22] Chenglu Li, Hangjun Che, Man-Fai Leung, Cheng Liu, and Zheng Yan. 2023. Robust multi-view non-negative matrix factorization with adaptive graph and diversity constraints. *Information Sciences* 634 (2023), 587–607.

[23] Xiang Li, Yao Wu, Martin Ester, Ben Kao, Xin Wang, and Yudian Zheng. 2022. SCHAIN-IRAM: An Efficient and Effective Semi-Supervised Clustering Algorithm for Attributed Heterogeneous Information Networks. *IEEE Transactions on Knowledge and Data Engineering* 34, 4 (2022), 1980–1992.

[24] Xiaoyang Lin, Renchi Yang, Haoran Zheng, and Xiangyu Ke. 2025. Spectral Subspace Clustering for Attributed Graphs. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*. 789–799.

[25] Kolby Nottingham Markelle Kelly, Rachel Longjohn. 2019. The UCI Machine Learning Repository. http://archive.ics.uci.edu/ml

[26] Gloriana Joseph Monko and Masaomi Kimura. 2023. SS-DBSCAN: Epsilon Estimation with Stratified Sampling for Density-Based Spatial Clustering of Applications with Noise. In *2023 International Conference on Automation, Control and Electronics Engineering*. 72–76.

[27] Tingting Qi, Xiangchu Feng, and Weiwei Wang. 2024. A game model for semi-supervised subspace clustering with dynamic affinity and label learning. *Signal Processing* 220 (2024), 109436.

[28] Alex Rodriguez and Alessandro Laio. 2014. Clustering by fast search and find of density peaks. *Science* 344, 6191 (2014), 1492–1496.

[29] Zhaiming Shen, Ming-Jun Lai, and Sheng Li. 2023. Graph-based Semi-supervised Local Clustering with Few Labeled Nodes. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. 4190–4198.

[30] Yifan Shi, Zhiwen Yu, Wenming Cao, C. L. Philip Chen, Hau-San Wong, and Guoqiang Han. 2021. Fast and Effective Active Clustering Ensemble Based on Density Peak. *IEEE Transactions on Neural Networks and Learning Systems* 32, 8 (2021), 3593–3607.

[31] Liping Sun, Fan Huang, Xiaoyao Zheng, Liangmin Guo, Qingying Yu, Zhenghua Chen, and Yonglong Luo. 2025. Density Peaks Clustering Based on Label Propagation and K-Mutual-Nearest Neighbors. *IEEE Transactions on Emerging Topics in Computational Intelligence* 9, 2 (2025), 1830–1842.

[32] Astrid Ustad, Aleksej Logacjov, Stine Øverengen Trollebø, Pernille Thingstad, Beatrix Vereijken, Kerstin Bach, and Nina Skjæret Maroni. 2023. Validation of an activity type recognition model classifying daily physical behavior in older adults: the HAR70+ model. *Sensors* 23, 5 (2023), 2368.

[33] Toon Van Craenendonck, Sebastijan Dumančić, Elia Van Wolputte, and Hendrik Blockeel. 2018. COBRAS: Interactive Clustering with Pairwise Queries. In *Advances in Intelligent Data Analysis XVII*. 353–366.

[34] Toon Van Craenendonck, Sebastijan Dumančic, and Hendrik Blockeel. 2017. COBRA: a fast and simple method for active clustering with pairwise constraints. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*. 2871–2877.

[35] Wen-Bo Xie, Bin Chen, Xun Fu, Jun-Hao Shi, Yan-Li Lee, and Xin Wang. 2024. Cost-effective hierarchical clustering with local density peak detection. *Information Sciences* 676 (2024), 120811.

[36] Wen-Bo Xie, Zhen Liu, Debarati Das, Bin Chen, and Jaideep Srivastava. 2023. Scalable clustering by aggregating representatives in hierarchical groups. *Pattern Recognition* 136 (2023), 109230.

[37] Wen-Bo Xie, Tian Zou, Tao Deng, Xuan-Lin Zhu, Qiu-Yu Wang Xun Fu, Bin Chen, and Xin Wang. 2025. A Robust and High-Efficiency Active Clustering Framework with Multi-User Collaboration. In *34th ACM International Conference on Information and Knowledge Management*. 561–568.

[38] Xiaojun Yang, Tuoji Zhu, Siyuan Peng, Feiping Nie, and Zhiping Lin. 2025. Semi-supervised pivotal-aware nonnegative matrix factorization with label and pairwise constraint propagation for data clustering. *Pattern Recognition* 157 (2025), 110933.

[39] Yan Yang, Tonny Rutayisire, Chao Lin, Tianrui Li, and Fei Teng. 2013. An Improved Cop-Kmeans Clustering for Solving Constraint Violation Based on MapReduce Framework. *Fundamenta Informaticae* 126, 4 (2013), 301–318.

[40] Jingxing Yin, Siyuan Peng, Zhijing Yang, Badong Chen, and Zhiping Lin. 2023. Hypergraph based semi-supervised symmetric nonnegative matrix factorization for image clustering. *Pattern Recognition* 137 (2023), 109274.

[41] Weizhong Yu, Liyin Xing, Feiping Nie, and Xuelong Li. 2023. An efficient semi-supervised balanced cut with hard pairwise constraints and partial labels. *Knowledge-Based Systems* 276 (2023), 110747.

[42] Guangjie Zeng, Hao Peng, Angsheng Li, Zhiwei Liu, Runze Yang, Chunyang Liu, and Lifang He. 2024. Semi-Supervised Clustering via Structural Entropy with Different Constraints. In *Proceedings of the 2024 SIAM International Conference on Data Mining*. SIAM, 208–216.

[43] Guangjie Zeng, Hao Peng, Angsheng Li, Jia Wu, Chunyang Liu, and Philip S. Yu. 2025. Scalable Semi-Supervised Clustering via Structural Entropy With Different Constraints. *IEEE Transactions on Knowledge and Data Engineering* 37, 1 (2025), 478–492.

[44] Chao Zhang, Deng Xu, Chunlin Chen, and Huaxiong Li. 2025. Semi-supervised Multi-view Clustering with Active Constraints. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1903–1912.

[45] Peng Zhou, Bicheng Sun, Xinwang Liu, Liang Du, and Xuejun Li. 2024. Active Clustering Ensemble With Self-Paced Learning. *IEEE Transactions on Neural Networks and Learning Systems* 35, 9 (2024), 12186–12200.

A Appendix

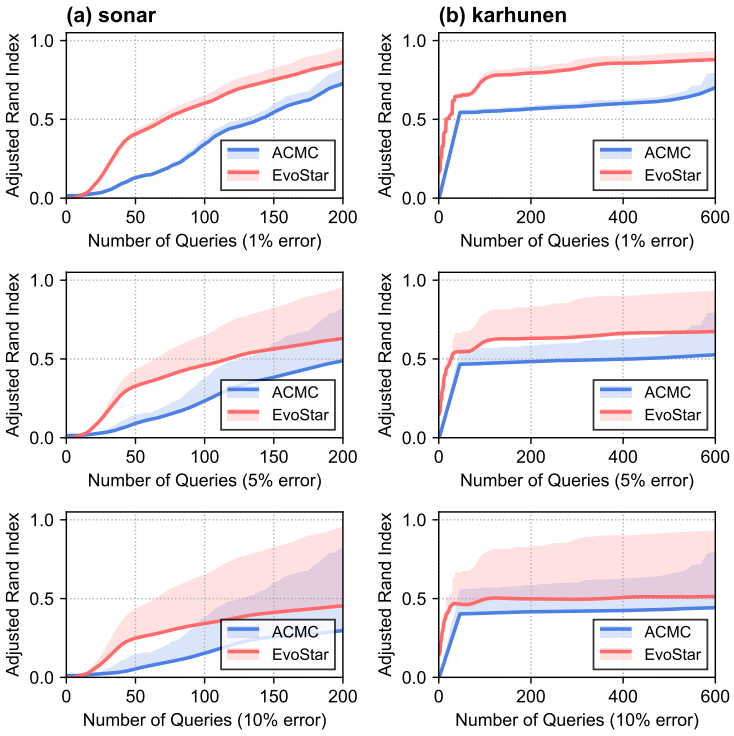


Figure 6: Performance comparison under 1%, 5%, and 10% oracle error rates for EvoStar and ACMC. EvoStar significantly outperforms ACMC across all noise levels. Notably, in the high-noise setting (10% error), while increasing noise inevitably impacts the absolute performance of both methods, EvoStar consistently maintains a distinct superiority over ACMC throughout the query process. In contrast, EvoStar demonstrates superior resilience, maintaining a steady upward trend in ARI and effectively mitigating the impact of noisy supervision. This confirms that EvoStar's evolving structure is robust not only to slight human errors but also to severe annotation noise.

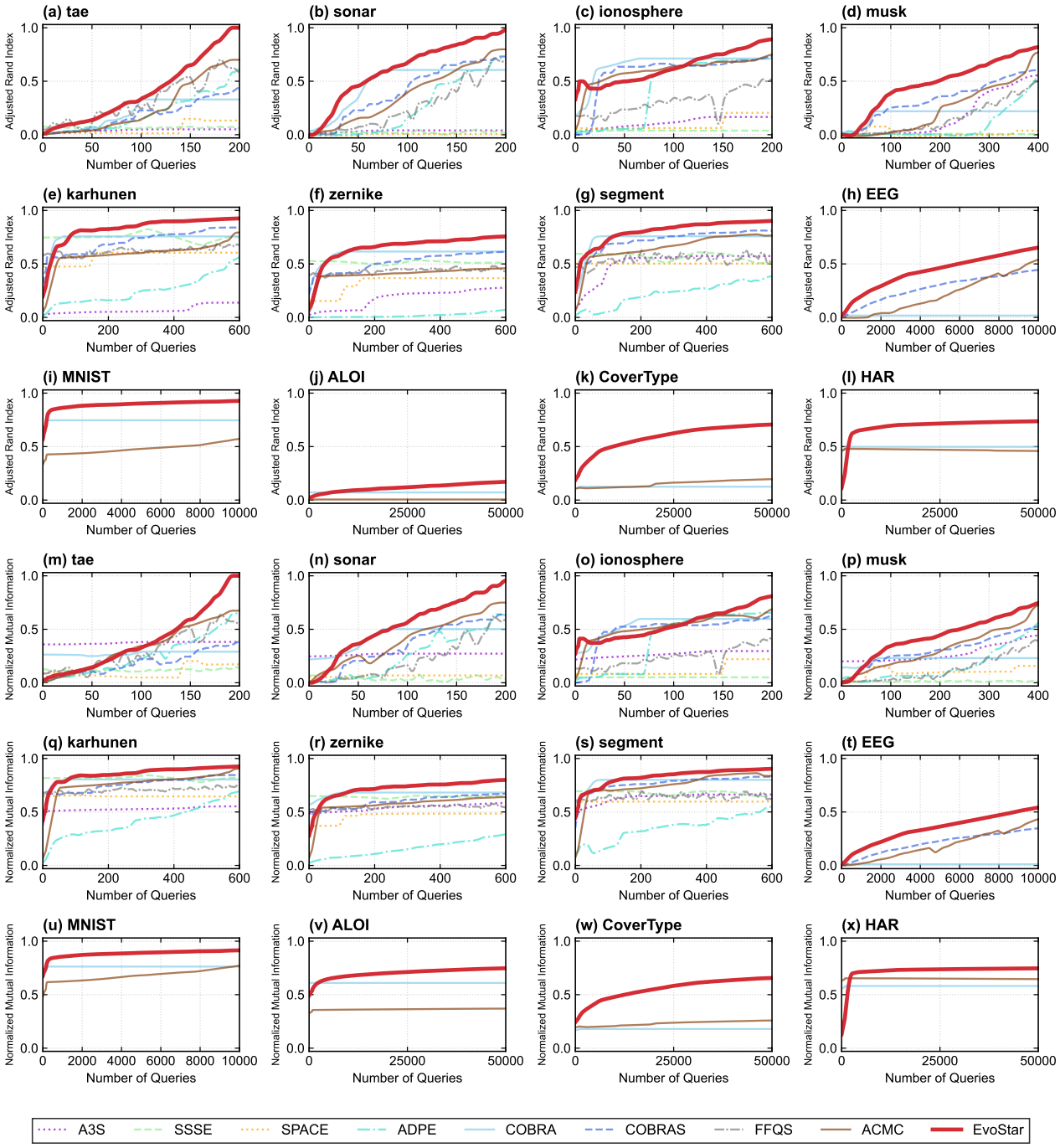


Figure 5: Evolution of clustering performance measured by Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) across 12 benchmark datasets demonstrates that EvoStar consistently outperforms baselines. A distinguishing feature of EvoStar is its rapid performance gain early in the semi-supervised learning process; on large-scale datasets like MNIST, CoverType, and HAR, it achieves superior results with minimal queries. Subsequently, EvoStar maintains a robust, monotonic improvement trajectory, converging to an optimal final solution and sustaining a clear performance gap over competitors (e.g., in musk and sonar). The experimental suite spans diverse data characteristics where most matrix-based methods fail to scale. In contrast, EvoStar exhibits exceptional scalability by efficiently navigating hierarchies without exhaustive pairwise comparisons. Ultimately, across all 12 datasets, EvoStar consistently avoids catastrophic performance degradation.